

Reviewer Pack — Minimal Coxeter Group Rank Lower Bound for Monotone Circuit ...

Entry 435c28c989af · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 19:28:24 UTC

Minimal Coxeter Group Rank Lower Bound for Monotone Circuit Size

Entry ID: 435c28c989af **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 19:28:24 UTC

1. Conjecture

Field A (mathematical branch): Coxeter / Hecke Algebras **Field B** (complexity object): Monotone Boolean Circuit Complexity

Statement:

[‘For any n -vertex monotone circuit C with size $s(C) = O(2^n)$, there exists a Coxeter group G associated with C such that the minimal rank of G is greater than or equal to $\log_2(n)$.’, ‘This lower bound holds for all circuits in C , and the minimal rank of G remains invariant under linear transformations.’, ‘If there exists an n -vertex monotone circuit C with size $s(C) = O(2^n)$ that can be associated with a Coxeter group G of minimal rank less than $\log_2(n)$, then $P=NP$.’]

Rationale (proposer’s reasoning):

[‘Coxeter / Hecke algebras provide a rich algebraic structure to study symmetries in combinatorial problems. Their ranks have been used as invariants in various areas, including geometry and representation theory.’, ‘A direct mapping from monotone circuits to Coxeter groups could reveal hidden structures that explain the complexities of monotone circuits.’, ‘Such an invariant would provide a new perspective on the hardness of monotone circuit complexity.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4928ac027aec7b2a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -vertex monotone circuit C with size $s(C) = O(2^n)$, if at least 25 of 30 independent runs yield a Coxeter group G associated with C such that its minimal rank is greater than or equal to $\log_2(n)$, the conjecture is supported. If any run produces a Coxeter group G with a minimal rank less than $\log_2(n)$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group" AND "monotone Boolean circuit complexity" - "minimal rank" AND "Coxeter algebra" AND "circuit size" - "P=NP" AND "monotone circuit" AND "Coxeter group rank"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_monotone_circuit(n):
    gate_type = random.randint(1, n)
    inputs = [random.randint(0, n-1) for _ in range(gate_type)]
    outputs = [random.randint(0, n-1) for _ in range(gate_type)]
    circuit = []
    for i in range(gate_type):
        gate = (inputs[i], outputs[i])
        circuit.append(gate)
    return circuit

def compute_coxeter_group_rank(circuit):
    # Construct a simple Coxeter group based on the circuit
    # This is a placeholder implementation; replace with actual computation
    rank = len(set(input for input, _ in circuit))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        circuit = generate_monotone_circuit(n)
        rank = compute_coxeter_group_rank(circuit)
        expected_rank = math.log2(n)
```

```

    if rank < expected_rank:
        return {
            "metric_name": "Coxeter Group Rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, rank={rank} < log2({n}) = {expected_rank}"
        }

    return {
        "metric_name": "Coxeter Group Rank",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) # Default to 1

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= math.log2(n_values[0])) / len(results)

    if all(r >= math.log2(n_values[0]) for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(r < math.log2(n_values[0]) for r in results):
        first_failing = next(i for i, r in enumerate(results) if r < math.log2(n_values[0]))
        print(f"RESULT: FALSIFIED counterexample=\"n={n_values[first_failing]}, rank<{math.log2(n_values[0])}\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'Coxeter Group Rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Coxeter Group Rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fbd87ca.py", line 67, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fbd87ca.py", line 55, in run_trial
    "metric_value": sum(results) / len(results),
                    ~~~~~^~~~~~

```

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot determine if the conjecture is supported or falsified based on the pre-registered support and falsification conditions. | next: Re-run the test without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16600
2	preregistration	ollama_remote	glm4:latest	0	0	9739
3	novelty	ollama_remote	glm4:latest	0	0	8491
4	novelty	ollama_remote	glm4:latest	0	0	8837
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12727
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8131
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10122
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8098
9	judge	ollama_remote	glm4:latest	0	0	27428

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110173 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/435c28c989af.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/435c28c989af.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/435c28c989af.tar.gz` (if generated)